

ALDIWAN

Restaurant Management System

Project Report

Object-Oriented Programming Course Project

Java Swing | MySQL | NetBeans

1. Introduction

ALDIWAN is a desktop restaurant management system developed in Java as an Object-Oriented Programming course project. It provides a single application through which restaurant staff can authenticate, take customer orders, process payments, manage takeaway deliveries, and, for managers, monitor staff and sales activity through a visual dashboard.

The application is built with Java Swing for the user interface, backed by a MySQL relational database for persistent storage of customers, menu items, orders, payments, and staff records.

1.1 Objectives

- Apply core object-oriented design principles (encapsulation, class composition, inheritance from Swing components) in a realistic, multi-screen desktop application.
- Model a small restaurant's operations, including ordering, payment, delivery, and staff administration, as a relational database and a matching set of Java classes.
- Build a role-based application where managers and staff see different views and capabilities.
- Provide a working example of a Java Swing application connected to MySQL via JDBC, packaged as a runnable NetBeans/Ant project.

2. System Overview

The system is organized around a login step that routes each user to one of two experiences based on their role:

- Manager: lands on the Dashboard, with charts summarizing staff and order activity, and full control over adding, updating, and deleting staff records.
- User (staff): lands on the Menu screen to take and process customer orders.

2.1 Core Features

- Role-based login with credentials validated against the person table.
- Manager dashboard with live bar and pie charts (JFreeChart), showing manager-to-employee counts, a breakdown of payment types, and orders categorized by Cash, Credit, and Takeaway.
- Staff record management from the dashboard: add, update, and delete person records.
- Menu browsing with item images and prices, and order building with selectable quantity.
- Three payment paths per order: Cash, Credit Card, and Takeaway/Delivery.
- Credit card payment capture, including card number, name on card, and expiry date, linked to the customer and the staff member who processed it.
- Takeaway/delivery handling, including customer name, address, phone, delivery fee, and amount due.

2.2 User Roles

Role	Entry Screen	Capabilities
Manager	Dashboard (Dash2)	View analytics charts; add/update/delete staff records
User (staff)	Menu	Browse menu, build orders, take payment (cash/credit/takeaway)

3. Architecture and Design

The application follows a straightforward layered structure typical of a small Swing/JDBC desktop project. Swing JFrame subclasses act as both the view and controller for each screen, and a single connection-holder class centralizes access to the database. There is no separate persistence or service layer; each screen issues its own SQL through JDBC directly.

3.1 Class Overview

Class	Responsibility
ALDIWAN	Application entry point; launches the LoginFrame.

LoginFrame	Authenticates a person against the database and routes to Dash2 (Manager) or Menu (User).
Dash2	Manager dashboard: analytics charts and staff (person) record management.
Menu	Order-taking screen: browse menu items, build an order, choose a payment method.
DeliveryForm	Captures takeaway/delivery details for an order (customer, address, fees, amount due).
CreditForm	Captures credit card payment details for an order.
DBConnection	Holds the single shared JDBC Connection, configured from db.properties.
PersonClass	Simple data holder for a logged-in person's name, role, password, and full name.
SessionPerson	Static utility holding the current session's user name, ID, and table state across screens.

3.2 Object-Oriented Concepts Applied

- Encapsulation: data-holder classes such as PersonClass group related fields together, and SessionPerson exposes controlled static accessors instead of raw shared state.
- Inheritance: every screen (LoginFrame, Dash2, Menu, DeliveryForm, CreditForm) extends javax.swing.JFrame, inheriting windowing behavior while specializing its own layout and logic.
- Single responsibility per class: connection handling, session state, and each screen's UI/logic are kept in separate classes rather than one monolithic class.
- Event-driven design: Swing's listener model (ActionListener callbacks on buttons/menus) drives control flow between screens, e.g. LoginFrame constructing and showing Dash2 or Menu on successful login.

3.3 Application Flow

- 1. ALDIWAN.main() starts the application and opens LoginFrame.
- 2. LoginFrame validates the entered username/password against the person table.
- 3. On success, the person's role determines the next screen: Dash2 for 'manager', Menu for 'user'.
- 4. From Menu, staff build an order and choose Cash, Credit (goes to CreditForm), or Takeaway (goes to DeliveryForm).
- 5. Order, payment, and delivery details are persisted to the corresponding MySQL tables.

4. Database Design

The database, aldiwandb, is a MySQL schema with six tables covering staff, customers, the menu, orders, credit payments, and takeaway deliveries. The schema and seed data are provided in ALDIWAN6/aldiwandb.sql.

4.1 Tables

Table	Key Columns	Purpose
person	person_id, name, role, password, fullname	Staff accounts used for login; role is 'manager' or 'user'.
customer	customer_id, customer_name, address, tel	Customer contact records.
menu_items	item_id, name, price	The restaurant's menu catalog.
orders	order_id, user_id, item_id, type, quantity, total_cost	An order line linking a staff member, a menu item, and a payment type.
paycredit	pay_id, customer_id, person_id, cash_withdraw, cardNo, name_on_card, expDate	Credit card payment records tied to a customer and the staff member who processed it.
takeaway	takeaway_id, order_id, customer_id, delivery_fees, customer_name,	Delivery/takeaway details for an order.

	customer_addr, customer_tel, amount_due	
--	--	--

4.2 Relationships

- orders.user_id references a staff member in person; orders.item_id references menu_items.
- paycredit.customer_id references customer; paycredit.person_id references the staff member (person) who took the payment.
- takeaway.order_id references orders; takeaway.customer_id references customer.

5. Technology Stack

Layer	Technology
Language	Java (JDK 8+)
UI Toolkit	Java Swing, built with the NetBeans GUI Builder
Charts	JFreeChart 1.5.3
Database	MySQL 8 (schema: aldiwandb)
DB Connectivity	MySQL Connector/J (JDBC)
Build Tool	Apache Ant (NetBeans-generated build.xml)
IDE	Apache NetBeans

6. Project Structure

ALDIWAN6/

- src/aldiwan/: Java source and NetBeans .form files for every screen
- Images/ and imagesSmall/: menu item photography (full-size and thumbnail)

- nbproject/: NetBeans project configuration
- aldiwandb.sql: MySQL schema and seed data
- build.xml: Ant build script

Generated build output (build/, dist/) is produced locally by the build and is not stored in the repository; see the README for build instructions.

7. Setup and Usage

7.1 Prerequisites

- Java JDK 8 or later
- MySQL Server
- NetBeans IDE (recommended) or Apache Ant

7.2 Steps

- 1. Import the schema: `mysql -u root -p < ALDIWAN6/aldiwandb.sql`
- 2. Copy `ALDIWAN6/src/db.properties.example` to `ALDIWAN6/src/db.properties` and set your local MySQL URL, username, and password.
- 3. Build and run via NetBeans, or from the command line: `cd ALDIWAN6 && ant run`

7.3 Sample Accounts

The seeded person table includes one manager account and several staff (user) accounts, listed in `aldiwandb.sql`, that can be used to log in immediately after set-up.

8. Conclusion

ALDIWAN delivers a complete restaurant management workflow, covering authentication, role-based views, order taking, multiple payment paths, delivery handling, and manager-facing analytics, all backed by a relational database. It puts object-oriented design into practice, including class decomposition, encapsulation, and inheritance from the Swing component hierarchy, in a working, runnable desktop application.

Possible Future Improvements

- Introduce a data-access layer to separate SQL from UI code, and use `PreparedStatement`s consistently to guard against SQL injection.
- Add input validation and clearer error dialogs across all forms.
- Replace the plain-text password column with a properly hashed password scheme.
- Add automated tests for order-total and payment calculations.